

Monica Joya

ITAI 3377: Artificial Intelligence for Edge and IoT Devices

A02: Deploying a Simple AI Model on a Simulated Edge Device - Reflective Journal

February 7, 2026

Reflective Journal

Introduction

This journal captures my experience working through each step of deploying a simple AI model on a simulated edge device for ITAI 3377. From setting up the environment to testing the model on a simulated Arduino through Edge Impulse, I documented what I did, what I observed, and what I took away from each part of the process.

Step 1: Set Up the Environment

I began by setting up the development environment on my Mac M4 using Visual Studio Code. I created a project folder (MINST_Edge_repo) and a Python virtual environment (edge_ai_env) to isolate dependencies. I installed TensorFlow 2.20.0 via `pip install tensorflow` within the virtual environment. For the Edge Impulse portion, the assignment directed me to download Node.js from the official website, but I chose to install it through Homebrew (`brew install node`) instead since it makes managing packages and future updates easier on macOS. This installed Node.js v25.6.0 and npm 11.8.0, then I installed the Edge Impulse CLI v1.37.2 using `npm install -g edge-impulse-cli`. I did not include Homebrew in my Simulation Documentation Report since it was just the method I used to install Node.js, not a tool used in the project itself.

Observation: The environment required tools from two different ecosystems—Python for model training and Node.js for Edge Impulse. Setting up each one came with its own steps: creating the virtual environment, installing TensorFlow and its dependencies through pip, then switching over to Homebrew for Node.js and npm before finally installing the Edge Impulse CLI. It was a multi-step process, but working through each layer helped me understand how all the pieces connect before writing a single line of model code.

Reflection: This step taught me the importance of proper environment setup before starting any AI project. Using a virtual environment kept my project dependencies clean. Choosing Homebrew over a direct download was a small decision, but I chose long-term maintainability even during initial setup. I also realized that edge AI development involves more than just Python—it requires familiarity with multiple package managers and toolchains.

Step 2: Prepare the Dataset

I loaded the MNIST dataset using `keras.datasets.mnist.load_data()`. The dataset contains 60,000 training images and 10,000 test images of handwritten digits (0–9), each 28×28 pixels in grayscale. I normalized pixel values from 0–255 to 0–1 using `x_train.astype("float32") / 255.0`, and reshaped the images to (28, 28, 1) using `np.expand_dims(x_train, -1)` to add the channel dimension required by the convolutional layers.

Observation: The preprocessing was straightforward with TensorFlow's built-in dataset loader. The normalization step is essential because neural networks perform better with input values in a small range. My code is slightly different from the instructor's provided snippets. I used `x_train.astype("float32") / 255.0`, like mentioned, instead of `x_train / 255.0` to make sure the data type is set to float32 before dividing, as well as `np.expand_dims(x_train, -1)` instead of `x_train.reshape((-1, 28, 28, 1))` to add the channel dimension. Both approaches do the same thing and give the same result, but from experience, the versions I used are common in TensorFlow documentation and make each step a little clearer to read.

Reflection: Although this dataset was ready to use, I understand that real-world data would require much more cleaning and preparation. This step reinforced how important data quality is to model performance.

Step 3: Train a Simple AI Model

I defined a CNN with the following layers: Conv2D (32 filters, 3×3, ReLU), MaxPooling2D (2×2), Flatten, Dense (128, ReLU), and Dense (10, softmax). The model was compiled using `model.compile(optimizer="adam", loss="sparse_categorical_crossentropy")` and trained using `model.fit(x_train, y_train, epochs=5)` with validation on the test set. Training completed in 18.57 seconds.

Training Results:

Epoch	Training Accuracy	Validation Accuracy
1	87.96%	96.69%
2	97.95%	98.23%
3	98.76%	98.53%
4	99.09%	98.23%
5	99.34%	98.59%

Observation: The model improved with each epoch, reaching 99.34% training accuracy and 98.59% validation accuracy by epoch 5. The close match between training and validation accuracy indicates the model was not overfitting.

Reflection: Watching the accuracy climb with each epoch made the iterative nature of neural network training feel tangible. The close match between training and validation accuracy gave me confidence that the model would generalize well to new data.

Step 4: Convert and Deploy the Model

TFLite Conversion:

I first saved the trained model using `model.save("mnist_model.h5")`, then converted it to TFLite format using `tf.lite.TFLiteConverter.from_keras_model(model)` followed by `converter.convert()`:

Metric	Value
Full Keras Model (mnist_model.h5)	8,360,808 bytes (8,164.85 KB)
TFLite Model (model.tflite)	2,778,572 bytes (2,713.45 KB)
Size Reduction	66.77%

Upload to Edge Impulse:

Following the assignment instructions, I attempted to upload the model using the CLI command: `edge-impulse-uploader --api-key <api-key> model.tflite`. The CLI returned an error stating it cannot handle .tflite files, as it only supports data file formats (.wav, .jpg, .csv, etc.). I then uploaded the model successfully through the Edge Impulse Studio web interface.

Observation: The 66.77% size reduction showed how effective TFLite conversion is for edge deployment. The CLI upload limitation was unexpected but the web interface worked as an alternative. My code includes an extra step not in the instructor's provided snippets — I used `model.save("mnist_model.h5")` to save the Keras model before converting it, while the instructor's code goes directly to `tf.lite.TFLiteConverter.from_keras_model(model)`. I included the save step to keep a copy of the full model as a backup before conversion, which is a common practice in case you need to go back and retrain or debug.

Reflection: This step taught me that model compression is a key part of edge deployment. You can't just take a full model and put it on a small device because the memory and processing power simply aren't there to support it. I also learned that troubleshooting tool limitations is a normal part of the development workflow, and when the CLI wouldn't accept the .tflite file, switching to the web interface and documenting why kept the project moving forward instead of getting stuck.

Step 5: Test and Validate the Model

I configured the model on Edge Impulse for image classification with input shape (28, 28, 1), pixel scaling 0–1, and 10 output labels (digits 0–9). I selected the Arduino Nano

33 BLE Sense (Cortex-M4F 64MHz) as the target device. Edge Impulse profiled the model using its EON™ Compiler (RAM optimized).

On-Device Performance:

Metric	Result
Target Device	Arduino Nano 33 BLE Sense (Cortex-M4F 64MHz)
Inferencing Time	3,695 ms
Peak RAM Usage	66.6 KB
Flash Usage	2.7 MB

I tested the model by uploading a handwritten digit image. The model correctly classified a handwritten “6” with 87% confidence. The only other notable prediction was “3” at 12%. All other digit classes received 0.00 probability.

Observation: The 3,695 ms inferencing time on the Arduino compared to near-instant results on my Mac made the performance trade-off of edge AI immediately visible. The 66.6 KB RAM usage and 2.7 MB flash usage confirm the model fits within the Arduino's constraints, but at a real cost to speed.

Reflection: This was the most insightful part of the exercise. The model works, but the device's limitations shape how and where it can actually be used. At nearly 4 seconds per prediction, this setup would be too slow for real-time object detection or live video classification where responses need to happen in milliseconds, but it could still work for tasks like reading handwritten forms or periodic sensor checks where a few seconds of delay is acceptable. That distinction between 'the model works' and 'the model works well enough for this use case' is something I would not have fully understood without running it on actual hardware.

Conclusion

From environment setup to edge deployment, this project connected a lot of moving parts into one complete pipeline. Working through each stage from training to converting to deploying and testing showed me how every decision early on affects what happens downstream. It was good to work through it end to end and walk away with something that actually runs on a constrained device, not just a model sitting in a notebook.